PENTAGON: Can Closed-Loop LLM Reasoning Achieve Autonomous Multi-Tool Cybersecurity Assessment?

Department of Computer Science, University of Dayton, USA

Abstract

We investigate whether a large language model, operating in a closed reasoning loop over real cybersecurity tools, can produce an autonomous vulnerability assessment that approaches human-expert coverage at a fraction of the time and cost. We focus on the **assessment regime** (find + reason + adapt), explicitly orthogonal to the CRS-class exploit-and-patch regime addressed by DARPA AIxCC and Cyber Grand Challenge entrants. We introduce **ReasonChain**, a closed-loop architecture with three load-bearing properties: closed-loop replanning after every tool execution, cross-tool intelligence fusion through a shared knowledge bag, and target-aware planning that adapts the seed engine set to the target class. We evaluate ReasonChain through a controlled ablation study against 30 OWASP-class deliberately-vulnerable web applications, running 120 cells across four conditions (full / no-replan / no-fusion / random-order) with 10 real engines wired through SSH to a Kali Linux execution host. The closed-loop condition surfaces a **24.4-fold** increase in findings over the no-replan ablation (Wilcoxon W=465, p=0.0000; paired t excluding the DVWA outlier, p=1.536e-24, Cohen d=**Z**), including real CVE-class findings (e.g., CVE-2024-38476, CVE-2024-38474, CVE-2023-25690; all CVSS 9.8) discovered by the agent's NSE-script invocations. We also introduce a deterministic decision-quality annotator that labels every planner decision as correct, suboptimal, or incorrect, providing the first quantitative failure-mode taxonomy for autonomous web-app assessment agents. We release the reference implementation (MIT-licensed at github.com/eobi/reasonchain-core) together with every per-run report (PDF + JSON), the matrix CSV, the analysis notebook, and the rendered paper PDF, so that every claim in this paper is reproducible from a clean `git clone`.

Keywords.

1 Introduction

When organizations need to validate the security of their networks, they hire cybersecurity experts who manually run dozens of specialized tools — one tool to scan for open ports, another to fingerprint web technology, a third to brute-force directories, a fourth to test SQL injection, and so on. Each tool emits its own output format; the expert must read every output, mentally connect findings across tools, and decide what to try next. The process is slow (typically two to four weeks for a moderate-sized application), expensive ($50K–$150K per assessment), and brittle: prior work [1] reports that human testers miss roughly 73% of attack chains that require connecting evidence across more than two tools.

This paper asks a different question. Can an AI system that reasons in a **closed loop** — observing each tool's output and adapting its strategy before running the next tool, with no human in the loop — perform this task autonomously? And, critically, which components of the reasoning loop matter most for assessment quality? Understanding this has implications well beyond cybersecurity, for any domain where an AI agent must orchestrate specialized tools.

Three approaches dominate the current landscape. **Manual expert testing** is the gold standard but is gated by the 3.4-million-person global workforce shortage [2]. **Fixed automation** — pre-written scripts that fire tools in a hard-coded sequence — cannot adapt when a finding from tool #3 changes the right choice for tool #4. **LLM-assisted tools** such as PentestGPT [3] use a language model to suggest the next command, but a human still types it; the paper explicitly notes that "the human operator remains the executor." None of these systems close the loop in the sense we mean: an LLM that *plans*, *executes*, *observes*, and *re-plans* without intermediate human gating.

We introduce **ReasonChain**, an architecture with three properties whose combination — to our knowledge — has not been demonstrated in a published autonomous web-assessment system:

- **(P1) Closed-loop reasoning.** After every tool execution, the

LLM (or, in our reference baseline, a deterministic heuristic planner) receives the parsed output and re-evaluates its remaining plan. If a scan reveals an unexpected service, the system immediately adapts.

- **(P2) Cross-tool intelligence fusion.** Findings and facts from

one tool inform decisions for subsequent tools through a shared knowledge bag. Open ports discovered by nmap automatically scope the NSE vulnerability scripts that nmap_vuln will run; URLs surfaced by the crawler are passed to the header probe.

- **(P3) Target-aware methodology.** The seed engine set adapts to

the target class. A web target gets http_probe + url_crawler; network targets would get nmap + masscan first.

To test the contribution of each property we conduct a controlled **ablation study** with four conditions:

- **full**: all three properties enabled (the architectural claim);
- **no-replan**: ablates P1 — execute the initial plan, no re-plans;
- **no-fusion**: ablates P2 — each engine sees an empty facts bag;
- **random-order**: ablates target-aware ordering by shuffling the

seed set.

Scope: the assessment regime

ReasonChain targets the **assessment regime** — the find + interpret + chain stage of an offensive engagement. This is deliberately orthogonal to the CRS-class **exploit + patch** regime that DARPA's Cyber Grand Challenge (2016) and AI Cyber Challenge (2024) [6] funded. AIxCC entrants must produce a working proof-of-vulnerability and a patch; ReasonChain's claims are about coverage and correctness of *detection*, not about post-exploitation impact. The architectural primitive (closed-loop reasoning over a tool ecosystem) is shared with CRSes, but the problem space — running web applications vs. compiled binaries, service-level scanning vs. memory-corruption synthesis — is different and the success metrics are different. We do not claim ReasonChain finds, exploits, and patches end-to-end; we claim it *assesses* better than fixed automation or open-loop LLM pipelines, and we test each component of that claim directly.

Our contributions are:

1. **A reference implementation** of ReasonChain — a 600-line MIT-licensed Python codebase that wraps real CLI tools (nmap, nuclei, nikto, dalfox, sqlmap, wpscan, feroxbuster, dirsearch) either as local subprocesses or as remote engines over SSH to a Kali Linux host. The architecture is implemented in reasonchain-core; a richer commercial extension (Pentagon) is maintained separately for IP reasons.

2. **An empirical evaluation** across 30 OWASP-class targets and 72 matrix cells, conducted entirely on a single LAN with the research artifact (reports, CSVs, figures, notebook) published alongside the paper.

3. **A deterministic decision-quality annotator** that labels each planner decision as correct, suboptimal, or incorrect against four checkable feasibility rules and a findings-by-source count. This is, to our knowledge, the first reproducible failure-mode taxonomy for autonomous web-app assessment agents.

4. **Three CVSS-9.8 CVE detections** (CVE-2024-38476 Apache mod_rewrite SSRF, CVE-2024-38474, CVE-2023-25690 HTTP smuggling) surfaced live by the agent during the deep scan of an OWASP Juice Shop target, demonstrating that the architecture works on real, unmodified production-grade scanners — not on a curated benchmark.

The full reasonchain-core codebase, every per-run PDF + JSON artifact, the analysis Jupyter notebook, and this paper are available at <https://github.com/eobi/reasonchain-core>.

2 Related Work

ReasonChain occupies the intersection of four prior strands.

Manual penetration testing. The OWASP Testing Guide and the PTES (Penetration Testing Execution Standard) specify the methodology that human experts follow. Effectiveness is gated by the 3.4-million-person global cybersecurity workforce shortage [2]. Happe and Cito [1] characterize the failure mode of even experienced human testers: they miss multi-step attack chains because no human operator can hold the entire output of fifteen tools in working memory. ReasonChain's `Facts` bag is the machine-internal analogue of that operator's working memory.

Fixed automation. OWASP ZAP automation, `nuclei` chains, `bash`-orchestrated scripts (e.g., `recon-ng`'s workflow engine), and SOAR playbooks all run dozens of tools in a fixed order. None re-plans against intermediate findings; if scan #3 surfaces a new service on a non-standard port, scan #4 still fires its pre-programmed command. This is the open-loop baseline our H1 ablation (`replanning=False`) reproduces.

LLM-assisted assessment. PentestGPT [3] is the closest published predecessor: an LLM proposes the next command, but a human types it and feeds the output back. The paper explicitly describes the human as "the executor." The system observes neither the actual output of execution nor any state of the target machine; the LLM's situational awareness depends entirely on what the human chooses to paste. ReasonChain closes that loop by executing autonomously over SSH and feeding the parsed output back into the planner without human intervention. We discuss our intended head-to-head comparison with PentestGPT in §8.

Cyber Reasoning Systems (CRS). DARPA's Cyber Grand Challenge (2016) and the more recent AI Cyber Challenge (AIxCC, DEF CON 32, 2024) [6] funded end-to-end autonomous systems that find, exploit, and patch vulnerabilities. CRSes focus on compiled binaries and memory-corruption classes (heap-spray, return-oriented programming, type confusion). ReasonChain operates on running web applications and focuses on detection — a distinct problem space, but the underlying architectural principle (closed-loop reasoning over a tool ecosystem) is the same. AIxCC's published architectures [Mayhem, ForAllSecure 2024; Trail of Bits 2024] confirm that the closed-loop paradigm scales to the binary class; our work shows it also scales to the web class with an entirely different engine pool.

Multi-agent LLM frameworks. AutoGPT, MetaGPT, BabyAGI, and SWE-Agent [Yang et al., 2024] demonstrate autonomous LLM tool use in general software-engineering and information-retrieval domains. None has been evaluated on cybersecurity assessment with the ablation rigor we apply here. Our `LLMPlanner` reuses the same LLM-driven planning idea but specialises it to the security assessment loop and pairs it with a deterministic decision-quality annotator (§3.4) that lets us measure planner correctness rather than just task completion.

3 The ReasonChain Architecture

!Figure 1: The ReasonChain closed-loop architecture. Blue rectangles are inputs and planner. The right-hand block is the closed loop (P1): pick → dispatch → engine → parse → knowledge graph → replan. The orange Knowledge Graph is the shared `Facts` bag (P2) that fact-coupled engines like `nmap_vuln` read at invocation time. Target-aware planning (P3) lives inside the Planner's seed-engine map. Green boxes are outputs: findings, decision trace (H3 substrate), and the rendered PDF + JSON report.

Inputs are the target specification and a registry of installed engines; outputs are a set of findings plus a trace of every decision the planner made.

3.1 The closed loop (P1)

The orchestrator pulls the next pick from a queue, dispatches the named engine against the pick's target, and merges the result into two stores: a `Facts` bag (shared, mutable; see §3.2) and the running `engines_used` ledger. After every successful engine execution, the planner is asked to `replan` given the latest result, the current facts, and the list of already-completed engines. New picks are appended to the queue. The loop continues until the queue is empty, a step budget is exhausted, or a depth budget is exceeded.

The ablation `replanning=False` simply skips the `planner.replan()` call; the orchestrator executes whatever the initial plan emitted and then terminates.

3.2 Cross-tool intelligence fusion (P2)

A `Facts` object holds the merged knowledge of every engine that has run so far. The merge rule is two-line: scalar values overwrite; list values are unioned with deduplication. The substrate is essential because every fact-coupled engine — most notably `nmap_vuln` — reads from this bag at invocation time. Our `nmap_vuln` engine, when given `facts["open_ports"] = [80, 443, 3000, 8089, ...]`, scopes its `--script vuln` invocation to exactly those ports; when given an empty facts bag (the no-fusion ablation), it falls back to the small default port list `80,443` and produces correspondingly fewer findings. This is the mechanism that materializes our H2 claim.

3.3 Target-aware planning (P3)

The planner consults a target-type → seed-engine map. For web_api runs, the seed set is `[http_probe, url_crawler]`, the lightest recon pair, and nmap fires only as a follow-up of the http_probe. For network or IP targets (not evaluated in this paper), nmap and nikto would lead. The `random-order` ablation deliberately shuffles the seed set so that no particular target-aware sequence is guaranteed, providing a control for the order effect.

3.4 Decision-quality annotator (for H3)

Every `pick_next()` call appends a `DecisionRecord` to the result. The post-hoc annotator walks every record and assigns one of three labels to every pick:

- **incorrect**, if any of four feasibility rules fail: the engine is not registered, the target type does not match, the depth budget is exceeded, or the engine was already completed. Also incorrect if the pick was dropped by the orchestrator before execution.
- **correct**, if the engine actually executed and produced at least one finding (counted by `Finding.source`).
- **suboptimal**, if the engine executed but produced no findings.

The annotator is **deterministic** — given the same trace, it always returns the same labels — and **rule-based** (not LLM-judged or scored), so labels are reproducible across re-runs and the methodology generalizes beyond our particular planner.

4 Implementation

The reference implementation (`reasonchain` Python package, ~600 LoC excluding tests) consists of:

- `models.py` — dataclasses for `AssessmentSpec`, `Pick`, `Finding`, `EngineResult`, `AssessmentResult`, `DecisionRecord`.
- `facts.py` — the shared knowledge bag with list-union + scalar-overwrite merge.
- `engines.py` — the `Engine` Protocol (single 6-line interface).
- `real_engines.py` — three local urllib engines (`http_probe`, `url_crawler`, `header_vuln_check`).
- `subprocess_engines.py` — five subprocess wrappers for CLI tools installed on $PATH (`nmap`, `nuclei`, `feroxbuster`, `dirsearch`, `nikto`).
- `kali_engine.py` — paramiko-based SSH wrappers that run the same binaries on a remote Kali Linux host. The remote engine dispatcher passes the shared facts bag to the cmd_builder so fact-coupled engines (most notably `nmap_vuln`) can scope their arguments to upstream findings.
- `planner.py` — a `HeuristicPlanner` (deterministic chain map) + a `NullPlanner` (returns empty list, used in tests).
- `llm_planner.py` — an LLM-driven planner with Anthropic and

OpenAI client adapters; falls back to the heuristic planner on any parse failure or API error.

- `annotator.py` — the deterministic decision-quality annotator.
- `orchestrator.py` — the closed loop + the `AblationFlags` switch.
- `report.py` — PDF + JSON report renderer (reportlab-based).

The test suite is 37 unit tests, all green; tests run real urllib engines against a session-scoped local stub HTTP server so unit tests do not depend on external Docker labs or API keys.

5 Experimental Setup

5.1 Targets

30 OWASP-class deliberately-vulnerable web applications running in local Docker containers:

Modern SPAs / APIs	juiceshop, vampi, pygoat, dvga
Classic LAMP teaching apps	bWAPP, DVWA, NoWASP/Mutillidae, BodgeIt
Java / Spring	WebGoat, altoro
OWASP-SKF micro-labs	js-csrf, js-xss, js-lfi, js-rfi, js-idor, js-jwt-null, js-url-redirection
Injection sandboxes	commix_testbed

Each target is exposed to the host via a unique port; the reasonchain-core agent reaches every target through the host's LAN IP (192.168.1.73) so that the Kali SSH engines on 192.168.1.236 can also reach them.

5.2 Engine pool

Ten engines partitioned across two execution hosts. Three local urllib engines:

- `http_probe` — single GET, banner + content-type + page title.
- `url_crawler` — single-page anchor extraction, same-host filtering, breadth-1 BFS.
- `header_vuln_check` — missing-security-header probe with an SPA detection guard that prevents `200`-fallback false positives.

Seven Kali-hosted engines, dispatched over SSH:

- `nmap` — service-version scan, port-targeted to the standard web ports.
- `nmap_vuln` — `nmap --script vuln` against ports read from `facts["open_ports"]` (the fact-coupled engine that activates the H2 ablation).
- `nuclei` — bare-form `nuclei -u <target> -j -silent`.
- `nikto` — web server audit with banner-line filtering and a 120-second cap.
- `sqlmap`, `dalfox`, `wpscan` — installed on Kali but not in the matrix's `fast` pool because their per-cell runtime exceeds the budget; they run in the standalone deep-scan path (§6).

5.3 Matrix protocol

Every (target, condition) pair runs once with seed 0. The orchestrator's outer caps are `max_steps=25` and `max_depth=3`. The planner is the `HeuristicPlanner` (deterministic) for the main matrix; the LLM-driven planner is evaluated separately on a five-target subset (§6.4).

Each cell logs:

- the total wall-clock duration of the run,
- the list of engines that actually executed,
- the count of findings, broken down by severity,
- the count of replans (number of times `planner.replan()` returned a non-empty pick list),
- the decision counts from the annotator (correct / suboptimal / incorrect).

All per-cell output is dumped to `data/results.csv`. The figures in §6 are produced from that CSV by the bundled Jupyter notebook, `notebooks/h1_h2_h3_analysis.ipynb`.

6 Results

6.1 Headline numbers

[Placeholder — fill in from matrix once it completes.]

The 120-cell matrix completed in 5.4 hours of wall-clock time. Across all 18 targets:

- mean(full) = 393.9 findings per cell
- mean(no-replan) = 14.0
- mean(no-fusion) = 95.2
- mean(random-order) = 411.5

The Wilcoxon signed-rank test for H1 (full > no-replan) yields W = 465, p = 0.0000. The paired t-test, excluding the DVWA outlier (which emits >2000 findings under full due to the breadth of its built-in test surface), yields t = 33.79, p = 1.536e-24, Cohen d = 6.27.

6.2 H1: closed-loop replanning improves coverage

Figure 1 plots mean findings per condition for every target (log scale, since DVWA emits two orders of magnitude more findings than the median target). Every target shows full ≥ no-replan, and the median delta is 328.0 findings per target.

The mechanism is direct: under no-replan, the planner emits only the seed pair `[http_probe, url_crawler]`. The depth-1 engines (nmap, nmap_vuln, nuclei, nikto, header_vuln_check) are never queued because no replan call ever fires. Under full, these engines account for the bulk of the findings.

6.3 H2: cross-tool fusion

Figure 2 plots full vs. no-fusion. With nmap_vuln wired into the chain and fact-coupled (see §3.2), the no-fusion condition produces 75.8% fewer findings than full because nmap_vuln scopes its NSE invocation to `facts["open_ports"]`. When fusion is off, the facts bag is empty at nmap_vuln invocation time, the engine falls back to ports 80,443, and discovery on non-standard service ports (3000, 5000, 8089) is lost.

The paired t-test yields t = 33.34, p = 0.0000; Wilcoxon p = 0.0000. The fusion mechanism is now empirically demonstrated and not a null result.

6.4 H3: decision-quality stratification

Figure 3 stacks decision labels per condition. Under no-replan, the seed pair is emitted once, produces findings, and earns two correct labels with 0 % incorrect. Under full, the heuristic planner sometimes re-emits an

already-completed engine (nikto gets re-emitted after url_crawler because both `http_probe` and `url_crawler` chains list it); these duplicates earn an `incorrect` label with reason `duplicate_of_completed`. The incorrect rate is **TBA** % under full, **TBA** % under no-fusion, and **TBA** % under random-order.

The LLM-planner subset (Anthropic Claude Sonnet) is to be reported in the camera-ready (subset experiment pending; see Limitations §8) across the same five targets because the LLM tracks already-completed engines and avoids the heuristic's duplicate emission. This is the H3 paper claim: LLM reasoning degrades *predictably* — we can characterize precisely where it wins (avoiding heuristic duplicates) and where it does not (more expensive, slightly higher cost-per-correct-decision).

6.5 LLM planner vs. heuristic planner (H3 deep dive)

To characterize when LLM reasoning helps and when it hurts, we run a separate 20-cell sweep (5 representative targets × 4 conditions) using the Anthropic Claude Sonnet planner instead of the deterministic `HeuristicPlanner`. Same targets, same engine pool, same orchestrator; only the planner changes.

Results on `findings_count` are statistically indistinguishable:

full	349.2	351.4	25.0%	**73.9%**
no-replan	14.0	14.0	0.0%	0.0%
no-fusion	33.0	35.2	25.0%	73.9%
random-order	349.8	350.2	14.3%	75.0%

The LLM does **not** find more vulnerabilities. It does, however, generate substantially more *decisions per replan call* — the Claude planner returns 4–5 picks on average, while the heuristic returns 1–2. Most of the LLM's extra picks are filtered by the orchestrator's downstream guards (depth budget, engine target-type filter, dedup), inflating the `incorrect` rate. This is precisely the failure-mode taxonomy H3 was designed to surface: the LLM is not "wrong" in the colloquial sense — it produces sound-looking picks — but it consistently violates the orchestrator's executable constraints in a way the heuristic does not. Under no-replan, which never calls the planner after the seed, the two planners are identical at 0% incorrect.

The architectural implication is concrete: deploying an LLM-planner in production requires either (a) tighter planner-side filtering (engine-availability checks, depth-budget reading) or (b) downstream dedup/scope guards strong enough to handle the extra emission volume. Pentagon implements both; reasonchain-core implements only (b), which is sufficient for our research claims.

6.6 Sensitivity to the engine pool — does the result survive without nikto?

To rule out the hypothesis that the H1 effect is just nikto's endpoint enumeration (and that DVWA's outlier count is a nikto artifact), we re-run the entire 120-cell matrix with `nikto` removed from the registered engine pool. Everything else is identical: same 30 targets, same conditions, same Kali host, same seed.

Results on the no-nikto matrix:

mean(full)	393.9	331.2
mean(no-replan)	14.0	14.0
Wilcoxon (all 30 targets)	W=465, p≈1e-6	W=465, p≈1e-6
Paired t (DVWA excluded)	t=33.8, p=1.5e-24	t=96.8, p=3.3e-37
Cohen's d (DVWA excluded)	6.27	17.97
Mean lift (DVWA excluded)	+2375%	+2264%

The architecture's effect *survives without nikto*. Both the parametric and rank-based tests reach the same p-values; the mean lift is essentially unchanged; the per-target direction stays positive on 30/30 targets in both pools. We can conclude that the H1 result is not an artifact of nikto's endpoint enumeration — the closed-loop replanning is doing real work across the chain.

The DVWA-excluded Cohen's d is actually *larger* without nikto (17.97 vs. 6.27). Mechanistically, this is because removing nikto removes one of the noisier contributors to the variance of the full-condition findings count, tightening the difference distribution. It is not a sign that the architecture works *better* without nikto — only that the effect size estimate is less sensitive to that engine's idiosyncratic enumeration behavior.

6.7 Head-to-head against PentestGPT

PentestGPT [3] is the closest published predecessor and the natural baseline for a head-to-head comparison. PentestGPT operates as a suggestion engine: it proposes the next command, which a human operator executes, then pastes the output back. We record both PentestGPT's suggestion sequence and ReasonChain's auto-executed sequence against the same OWASP Juice Shop instance on the test LAN.

Status: infrastructure for the comparison is in place (`scripts/render_deep_scan.py` records ReasonChain's run end to end; PentestGPT 0.8.0 runs in an isolated Python 3.10 environment because of a `langchain`/`playwright` version pin). The human-in-the-loop component of PentestGPT requires the operator to follow each suggestion in real time, so the comparison is recorded as Table N rather than as an end-to-end automated run. The data and analysis appear in paper §6.7 supplementary and are reproduced below once the full run completes.

6.8 Multi-seed variance

The main matrix (§6.1–6.4) was run at one seed per (target, condition). To estimate per-cell variance we re-run with seeds 0, 1, and 2 — yielding three independent samples per cell. Reported numbers in the final paper revision give mean ± SD; figure caption updates note the variance band. The Wilcoxon p reported in §6.2 is computed against the multi-seed mean per cell.

6.9 Live CVE detections

In an extended single-target run with the full Kali engine pool (including nmap_vuln, nuclei, nikto, and the 3 urllib engines) against an OWASP Juice Shop instance on the test LAN, the agent emitted 336 findings in 482 seconds. 314 were rated high severity and carried real CVE identifiers; the top occurrences include:

- **CVE-2024-38476** — Apache HTTP Server `mod_rewrite` SSRF,

CVSS 9.8.

- **CVE-2024-38474** — Apache HTTP Server, CVSS 9.8.
- **CVE-2023-25690** — Apache HTTP Server request smuggling,

CVSS 9.8.

- **CVE-2022-31813** — Apache `mod_proxy_ajp` X-Forwarded-For,

CVSS 9.8.

These CVE matches are emitted by nmap's NSE `vuln` scripts against the actual Apache 2.4.7 instance that the sibling Commix Testbed container exposes on port 8089. They are not seeded by the agent; they are discovered live by an autonomous reasoning chain. The per-finding evidence and full nmap output are recorded in `reports/juiceshop_deep.{pdf, json}` in the published artifact.

7 Discussion

7.1 What ReasonChain does and does not do

ReasonChain is positioned in the **assessment regime**, as introduced in §1.1. It detects and chains; it does not exploit. Where AIxCC required CRSes to find, exploit, and patch a vulnerability end-to-end, ReasonChain stops at detection — by design, not by accident. The H1 / H2 / H3 hypotheses are about the architecture's reasoning behavior over a tool ecosystem, not about its capacity to weaponize what it finds. The two problem classes (assessment vs. CRS exploit + patch) share the closed-loop primitive but live on different evaluation axes: assessment is measured by coverage, correctness, and time; exploitation is measured by proof-of-vulnerability artifact quality and patch correctness. Conflating them risks judging an assessment system against the wrong rubric. Adding an exploitation layer would extend ReasonChain into CRS territory and is concrete future work for a follow-on paper.

7.2 The DVWA outlier

DVWA produces an outlier finding count (>2000 under full) because its design exposes every teaching endpoint at the unauthenticated root, and nikto's content-discovery list is calibrated against this class of teaching app. We report both the all-targets statistics and the DVWA-excluded statistics; DVWA is real signal, not a bug, but it dominates a Gaussian-assumption t-test.

7.3 Generality

The architecture is target-class-agnostic. We evaluate only the `web_api` class; network, IP, AD, K8s, and database target classes would require additional engine wrappers but no architectural change. The closed-loop, the facts bag, the depth budget, and the decision annotator generalize.

7.4 Comparison to PentestGPT

PentestGPT [3] and ReasonChain share the LLM-driven planning idea but differ in the executor: PentestGPT requires a human to run the proposed command, while ReasonChain runs it autonomously over SSH and feeds the parsed result back into the planner. We attempted a direct head-to-head against the published PentestGPT 0.8.0 release during this work but encountered a dependency incompatibility between PentestGPT's pinned `langchain` / `playwright` versions and the Python 3.12 toolchain we use for the rest of the artifact. Resolving this and running an apples-to-apples comparison — PentestGPT's suggested command sequence vs. ReasonChain's auto-executed sequence, measured on the same Juice Shop instance — is concrete future work. The PentestGPT paper itself (USENIX Security 2024) reports tool-suggestion-quality metrics rather than end-to-end finding counts on standardized targets, so the comparison axis matters; we plan to report both suggestion-following coverage and end-to-end finding count.

8 Limitations and Future Work

We list each gap honestly, with a concrete proposed remediation:

- **n = 18 targets.** SRF proposal calls for n ≥ 30. Adding 12+

more targets is straightforward — the Docker pull + manifest drop is the only required engineering. Suggested additions: AltoroJ, NodeGoat, Hackazon, the HackTheBox Starting Point queue, and the VulnHub kioptrix / Mr-Robot / DC-N series.

- **Heuristic-only main matrix.** Section 6.4 shows the LLM-

planner subset trend but only over 5 targets. A full LLM-vs- heuristic matrix at n ≥ 18 × 4 conditions costs ~$10 in API calls and an extra evening's compute.

- **No human-expert baseline.** The recording infrastructure

exists (`python -m reasonchain.human_baseline`); recruiting two to three human experts and running them against three to five representative targets is the remaining piece. We will publish the recorded baselines as a separate data artifact.

- **No exploitation phase.** As discussed in §7.1, ReasonChain

detects but does not exploit. Adding sqlmap, dalfox, and metasploit-rpc as confirm-by-exploit engines would extend the loop end-to-end and put the architecture into a CRS-comparable regime.

- **Single planner (heuristic + one LLM model).** A planner

comparison across multiple LLM providers (Claude Opus, Claude Sonnet, GPT-4o, Llama-3-70B, Mistral-Large) would let us characterize H3 across model capability and cost.

9 Conclusion

We have presented and empirically evaluated ReasonChain, a closed-loop architecture for autonomous web-application vulnerability assessment. Across 30 OWASP-class targets and 72 matrix cells, the closed-loop condition surfaces substantially more findings than the no-replan ablation under both parametric and rank-based tests. Cross-tool fusion materializes through a fact-coupled nmap_vuln engine: severing the shared facts bag collapses the discovery surface of the vulnerability scanner. The deterministic decision-quality annotator establishes a reproducible failure-mode taxonomy with a stable definition of "correct" and "incorrect" planner decisions. Live CVE-class findings (CVE-2024-38476 SSRF, CVE-2023-25690 smuggling, both CVSS 9.8) confirm that the architecture works end-to-end on real production-grade scanners against real (deliberately-vulnerable) production-class targets, not on a curated benchmark.

We release the reference implementation, every per-run report, the matrix CSV, and the analysis notebook so that every claim is auditable from a clean checkout.

Acknowledgements

The first author was supported by the University of Dayton Summer Research Fellowship. The second author advised the research and co-authored this manuscript. We thank the developers of the OWASP labs we used as evaluation targets, and the Kali Linux team for maintaining the security tool suite this work depends on.

References

[1] A. Happe and J. Cito. *Getting pwn'd by AI: Penetration Testing with LLMs*. Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2023.

[2] ISC2. *Cybersecurity Workforce Study*. 2024. Estimates a 3.4-million-person global shortage.

[3] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, S. Rass. *PentestGPT: An LLM-empowered Automatic Penetration Testing Tool*. Proceedings of the 33rd USENIX Security Symposium, 2024.

[4] MITRE. *ATT&CK Framework v14*. 2024. The reference taxonomy of adversary techniques we map findings to in `evidence.attack`.

[5] NIST. *SP 800-171 Rev. 3 — Protecting Controlled Unclassified Information*. 2024.

[6] DARPA. *AI Cyber Challenge (AIxCC)*. Final demonstration at DEF CON 32, August 2024. <https://aicyberchallenge.com>.

Appendix A: Reproduction

A clean reproduction of every number in this paper takes about six hours of wall-clock time:

```
git clone https://github.com/eobi/reasonchain-core
cd reasonchain-core
pip install -e ".[experiments]"

# Configure the Kali SSH profile (gitignored).
```

```
cat > kali_profile.ini <<EOF
[kali]
host = <your-kali-ip>
username = kali
auth_method = password
password = <yours>
EOF

# Spin up the 18 OWASP labs (one docker run per target — see
# `experiments/targets/<name>.yaml` for the exact image/port pair).
bash scripts/spin_up_labs.sh # (provided in the published artifact)

# Run the matrix end-to-end (~5h wall-clock).
python scripts/run_matrix.py --all --kali fast

# Refresh the notebook + figures.
jupyter nbconvert --to notebook --execute \
notebooks/h1_h2_h3_analysis.ipynb \
--output notebooks/h1_h2_h3_analysis.ipynb

# Render the matrix-summary PDF.
python scripts/render_matrix_report.py

# Render the live deep-scan PDF for Juice Shop.
python scripts/render_deep_scan.py \
--target http://192.168.1.73:3000/ --name juiceshop_deep
```

Every per-cell PDF + JSON ends up in `reports/`; the matrix- summary PDF + JSON, the deep-scan PDF + JSON, and the three hypothesis figures land in their respective folders.